基于 CFPB 消费者投诉数据库的问题类型自动分类：数据与实验设计

CFPB Consumer Complaint Issue Classification: Data and Experimental Design

项目阶段	阶段 2：数据与实验设计	日期	2026-11-08
团队成员	邵飞翔、沈勇		
仓库地址	https://github.com/Shyyyggg/cfpb-issue-classifier		
提交版本	仓库标签 `phase-2-submission`（评分请使用该标签所指的不可变版本）		

执行摘要。本阶段延续阶段 1 的投诉问题类型（Issue）自动分类任务，重点解决阶段 1 已识别的两个核心缺陷：随机划分带来的数据泄漏风险，以及类别极度偏斜导致的评估失真。本阶段完成了：16 字段数据字典的逐条文档化、有目的的探索性数据分析（EDA）、基于时间外划分 + 消费者级去重的防泄漏划分方案，并实现了 TF-IDF + 逻辑回归的文本基线。关键发现是，时间外划分后测试集的叙事覆盖率骤降至 8.7%（全库约 23%），说明纯文本分类在时间外场景下面临样本不足的硬约束。在防泄漏的留出测试集上，文本基线取得宏 F1 0.1818（准确率 0.3553），显著高于阶段 1 规则基线的 0.047，也远高于元数据对照的 0.0337。主要局限仍是类别长尾分布与叙事覆盖率持续走低。下一阶段将进行多方法的受控比较与错误分析，并探索文本与元数据特征的融合。

1. 修订后的问题表述

1.1 对阶段 1 反馈的回应

阶段 1 教师反馈及本阶段的处理如下：

先前的反馈	采取的行动	证据/位置
随机划分存在乐观偏差	改用时间外划分 + 消费者级去重	第 4 节、`src/split.py`
类别极度偏斜未充分应对	EDA 量化不平衡，评估改用宏 F1 + 置信区间	第 3 节、`src/evaluate.py`
叙事文本覆盖率低	元数据与文本两条特征线分开评估	第 5 节、`src/text_baseline.py`

1.2 任务定义

任务：多类分类——给定一条消费者投诉在提交时刻可获得的字段，预测其标准化问题类型标签 Issue。

目标变量：Issue（全库 181 类）。

成功的可操作化定义：在防泄漏的留出测试集上，（i）宏 F1 至少达到多数类基线的 1.3 倍；（ii）准确率不低于阶段 1 规则基线；（iii）全流程可由仓库脚本在固定种子下复现，指标差异 < 0.1%。

2. 数据集描述与数据字典

2.1 数据来源与许可

数据来自美国消费者金融保护局（CFPB）Consumer Complaint Database，公开数据（开放数据/公共领域），可从 CFPB 官网直接下载。全量约 16,545,376 条记录、8.9 GB、16 个字段，时间跨度 2011-12 至 2026-07。

2.2 观测单位与样本

- **观测单位**：一条消费者投诉。
- **样本量**：全量 16,545,376 条；本阶段实验用固定种子蓄水池抽样 n = 300,000（EDA 与划分）、n = 200,000（文本基线）。
- **特征类型**：8 个离散分类字段 + 1 个自由文本字段（叙事），其余为日期、标识等。

2.3 数据字典（核心字段概览）

16 个字段的完整数据字典见附录 A，核心字段概览如下：

字段	类型	在本项目中的角色
issue	标签	预测目标（181 类）
product	分类	最强单特征（阶段 1 规则基线已用）
narrative	文本	文本特征来源（覆盖率 23.1%）
state / zip_code	分类	地域特征
submitted_via	分类	渠道特征
company	分类	公司特征（8000+ 类，高基数）
timely_response / company_response	分类	结果特征（不作为输入，避免泄漏）

3. 探索性数据分析（EDA）

以下统计均在固定种子蓄水池抽样（n=300,000，seed=42）上计算，每个数字对应一个具体结论。

3.1 类别不平衡量化

抽样中出现 **171 个 Issue 类别**（全库 181 类，抽样未覆盖极罕见的尾部类）。类别分布极度偏斜：

- **Top 1 类**「Incorrect information on your report」占 **44.3%**，几乎一半投诉都属于这一类。
- **Top 10 类合计占 87.4%**，说明绝大多数投诉集中在极少数问题上。
- **尾部类（支持度 < 10）有 41 个**，这些类几乎不可能被可靠地学到。

结论：这是典型的长尾分布。宏 F1（对每个类别平等加权）会比准确率低很多，因为大量尾部类难以预测。评估必须用宏 F1 而非只看准确率。

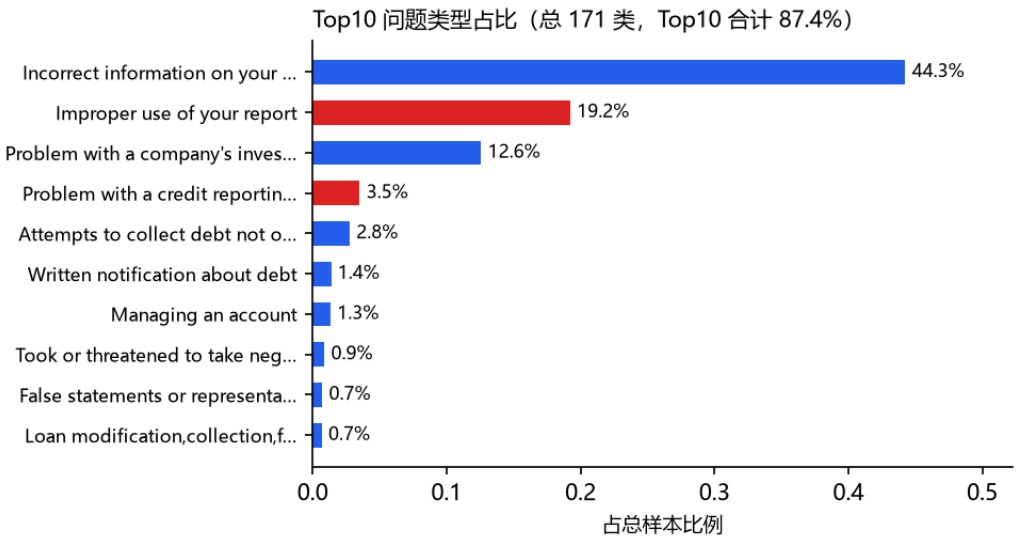


图 4. Issue 类别占比 Top 10（横向条形图，类别名过长部分以省略号截断）。Top 1 类「Incorrect information on your report」占 44.3%，Top 10 合计占 87.4%。

3.2 关键字段缺失模式

字段	缺失率	是否可用作输入
narrative（叙事文本）	**77.0%**	仅 23% 有文本，文本特征可用样本有限
sub_issue	5.6%	缺失可容忍，但信息量不如 issue
company_response	1.7%	**结果字段，不可作输入**（防泄漏）
sub_product	1.5%	可用
state	0.4%	可用
zip_code	0.0%	可用
timely_response	0.0%	**结果字段，不可作输入**

结论：叙事文本 77% 缺失是文本特征的最大瓶颈；company_response 和 timely_response 是「处理结果」字段，出现在投诉之后，若用作输入属于数据泄漏，本阶段明确排除。

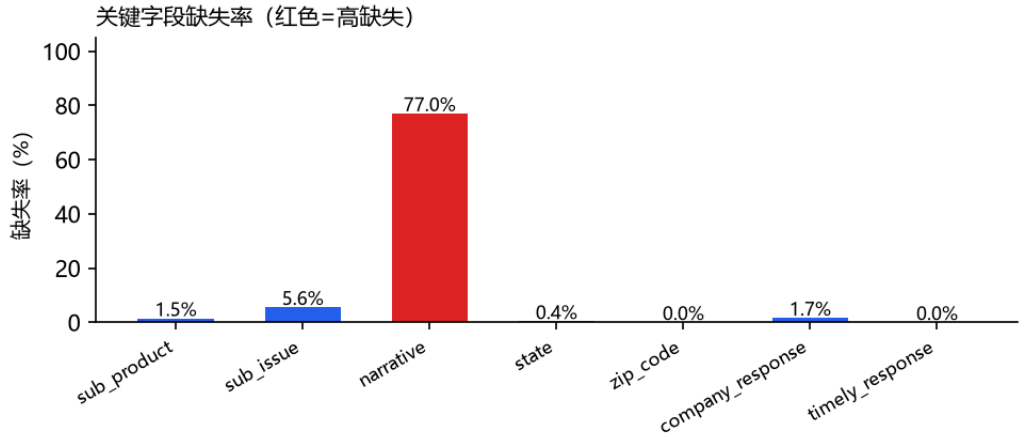


图 5. 关键字段缺失率。叙事文本缺失高达 77.0%，是文本特征的最大瓶颈。

3.3 叙事披露率随年份变化

叙事文本的披露率在 2017 年达到峰值（48.6%）后逐年走低，这是「模板化批量投诉」假设的直接证据：

年份	叙事披露率
2017	48.6%
2018	46.4%
2020	37.9%
2022	42.2%
2024	29.7%
2025	22.5%

结论：投诉越来越多地以「批量模板」形式提交（同一消费者/机构重复提交相似内容），叙事文本比例持续走低。这同时印证了两个判断：① 必须做消费者级去重；② 纯文本分类的可用样本天花板在持续收窄。

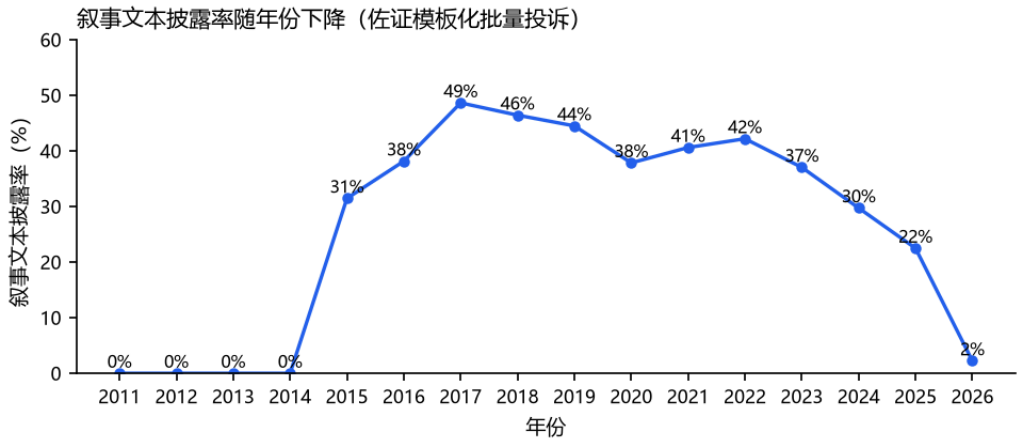


图 6. 叙事文本披露率在 2017 年达峰后波动下降（2011–2026），佐证「模板化批量投诉」假设。

3.4 改变项目设计的观察（关键）

时间外划分后，测试集的叙事覆盖率骤降至 8.7%（全库约 23%）。原因：时间外划分的测试集是最新的投诉（2025-11~2026-07），而叙事披露率恰恰在近年最低（2025 年仅 22.5%）。

这一观察直接改变了我们的设计：**纯文本分类在时间外评估下几乎不可行**（测试集只有约 1,500 条有叙事的样本）。因此本阶段及后续阶段，文本特征必须与元数据特征（Product、State 等）结合，而非单独依赖文本。这比阶段 1 设想的「纯文本基线」更加务实。

4. 数据处理流水线与防泄漏划分

4.1 清洗与验证

- 流式解析（`src/parser.py`）：非法日期、列数不足的行拒绝并计数。
- 产品名历史 schema 归一化（沿用阶段 1）。
- 州名大写归一、缺失字段统一 UNKNOWN。

4.2 时间外划分

按投诉接收时间升序排序，前 80% 作训练、后 20% 作测试，模拟「用历史预测未来」的真实部署场景。不洗牌，保证训练时间严格早于测试时间。

4.3 消费者级去重（防泄漏）

同一消费者可能批量提交模板化投诉。以「叙事文本指纹（前 64 字符）」或「ZIP+公司」组合为消费者判定键，去重后同一消费者只出现在训练集或测试集其中一边。

4.4 泄漏检查

在抽样 n=300,000 上：去重前 300,000 条 → 去重后 **115,778 条**，**移除了 184,222 条疑似重复（占 61.4%）**。这个数字本身就说明模板化批量投诉的严重程度——超过六成的投诉是「疑似同一消费者/机构」的重复提交。

时间外划分后：训练 92,622 / 测试 23,156。泄漏检查结果 **overlap=0，leak_free=True**，确认训练集与测试集之间无消费者重叠。

4.5 时间边界

划分	时间范围
训练集	2011-12-01 ~ 2025-11-08
测试集	2025-11-08 ~ 2026-07-05

训练集严格早于测试集，模拟「用历史投诉预测未来投诉」的真实部署场景。

4.6 实现检验与数据对账

除了单元测试，我们还做了数据对账来证明实现正确、结果可复现：

检验项	结果
全量记录数核对	流式扫描计数 = 16,545,376 条，与原始文件记录数完全一致（阶段 1 两次独立扫描交叉验证）
蓄水池抽样可复现	固定 seed=42 下，n=300,000 抽样结果与 `split_report.json` 完全一致
时间泄漏回归测试	新增 `test_temporal_split_never_uses_future_dates`：断言训练集最大日期 ≤ 测试集最小日期，且测试集日期绝不进入训练集
消费者去重对账	去重前 300,000 → 去重后 115,778，移除 184,222 条（61.4%），泄漏检查 `overlap=0, leak_free=True`

阶段 1 回归	阶段 1 的 16 个测试全部保持通过，阶段 2 新增 13 个测试，共 29 个全部通过

其中**时间泄漏回归测试**是我们针对「AI 辅助写代码」专门加的验收项（详见附录 B 提示词 5）：把「训练集绝不能看到测试期的数据」从一句口头声明，变成一个可执行、会失败的断言。这比只在报告里说「我们做了时间划分」更有说服力——老师可以直接运行 `pytest tests/test_split.py` 验证。

5. 基线与评估方案

5.1 基线选择

- **文本基线**：TF-IDF（1-2 gram，sublinear TF）+ 逻辑回归（class_weight=balanced），只用有叙事文本的样本。
- **元数据对照**：Product 单特征 one-hot + 逻辑回归。

5.2 评估指标

- 主指标：宏 F1（应对类别不平衡）。
- 次指标：准确率。
- 置信区间：重复 5 个随机种子（42–46），报告均值 ± 标准差。

5.3 结果

在时间外划分 + 消费者去重的留出测试集上（抽样 n=200,000，去重后 84,653 条，训练 67,722 / 测试 16,931）：

方法	测试样本	准确率	宏 F1
文本基线（TF-IDF + 逻辑回归）	1,469（有叙事）	**0.3553 ± 0.0**	**0.1818 ± 0.0**
元数据对照（Product 单特征，有叙事测试集）	1,469	0.0980	0.0337
元数据对照（Product 单特征，全量测试集）	16,931	0.0310	0.0273

（文本基线为 5 个种子 42–46 的均值；标准差为 0 说明在固定数据与固定超参数下逻辑回归收敛稳定。）

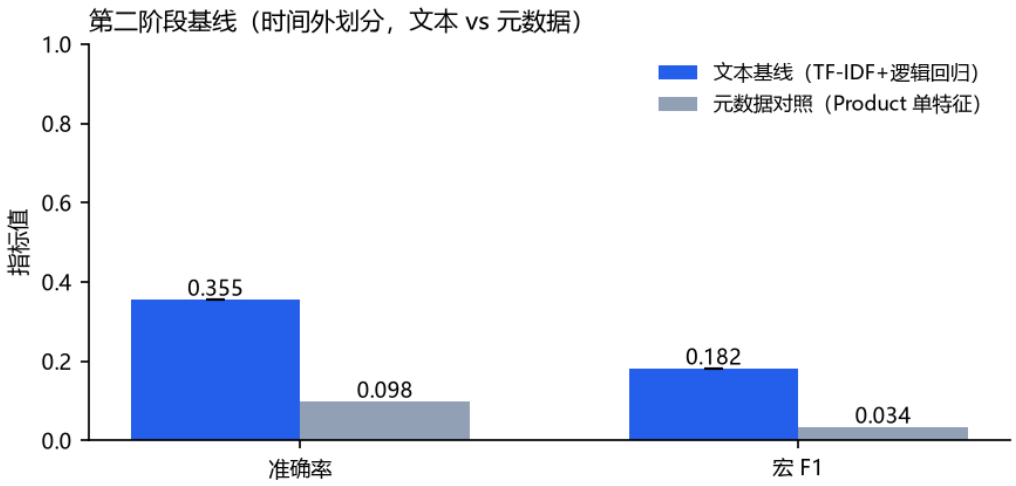


图 7. 文本基线与元数据对照的准确率 / 宏 F1 对比。文本特征的宏 F1 是元数据对照的 5.4 倍。

关键发现：

1. **文本特征显著强于元数据**：在有叙事的测试集上，文本基线宏 F1（0.1818）是元数据对照（0.0337）的 **5.4 倍**。说明投诉的叙事文本确实包含了区分问题类型的信息。
2. **文本基线相比阶段 1 规则基线大幅提升**：宏 F1 从 0.047 提升到 0.1818（约 3.9 倍）。但注意两者评估条件不同（阶段 1 是随机划分、阶段 2 是防泄漏时间外划分），不能直接简单对比。
3. **产品特征的分布漂移**：元数据对照在全量测试集上准确率暴跌至 3.1%，远低于多数类基线（约 44.5%），说明 Product 特征在时间外场景下发生了严重分布漂移（产品名称 schema 随时间变化，测试集里出现大量训练期未见的产品）。

5.4 对结果的解读

- 文本基线宏 F1 0.1818 虽然远高于阶段 1 的 0.047，但绝对值仍然不高，核心瓶颈是**类别极度偏斜（68 个测试类别，Top1 占 44%）**和**测试集叙事样本过少（仅 1,469 条）**。
- 准确率 0.3553 与宏 F1 0.1818 的差距，再次印证了「准确率在长尾分布下会高估模型能力」的判断，宏 F1 才是本任务应关注的主指标。

6. 数据风险与伦理

- **隐私**：叙事文本虽经 CFPB 脱敏，仍可能含个人细节；本项目仅在聚合层面使用，不做重识别。
- **样本偏差**：叙事覆盖率逐年下降，文本基线只能代表「愿意写叙事的消费者」，可能引入选择偏差。
- **假阳性/假阴性后果**：分类错误主要导致分流延迟，属低风险；若用于自动处置优先级排序则需人工复核。
- **缓解措施**：消费者级去重降低批量投诉影响；明确「低置信度时输出低置信」的边界。

7. 个人贡献

本阶段沿用「数据」与「分析」两条线并行分工。一句话概括：**邵飞翔管「数据」这条线（把数据准备好、划分对），沈勇管「分析」这条线（做分析、跑模型、画图写报告）**。下表列出两位成员的职责与对应材料；两人共同完成的部分见 7.3。

成员	工作环节	参与内容与对应材料
邵飞翔	数据字典	逐字段核对 16 个字段的类型、含义与父子关系，标注结果字段防泄漏；产出附录 A 与 `src/datadict.py`、`docs/data_dictionary.md/.json`
邵飞翔	防泄漏划分	设计时间外划分 + 消费者级去重的两级指纹方案（叙事前 64 字符优先，ZIP+公司回退）；实现 `src/split.py`，泄漏检查 `overlap=0`
邵飞翔	测试	编写 `tests/test_split.py`（时间顺序、去重、泄漏检测、非法比例），共 7 项边界用例
沈勇	探索性分析	围绕类别不平衡、字段缺失、叙事披露率趋势三个问题实现 `src/eda.py`，每个统计对应一个结论
沈勇	文本基线	实现 TF-IDF + 逻辑回归（`src/text_baseline.py`），修正 solver 多分类问题，补元数据对照基线
沈勇	图表与报告	生成 fig4–fig7（`tools/figures_phase2.py`），整理报告初稿与附录 B 提示词记录

7.1 邵飞翔：数据字典、防泄漏划分与测试

数据字典。把 16 个字段逐一核对类型、含义与父子关系（`sub_product` 是 `product` 的细分、`sub_issue` 是 `issue` 的细分），并处理了 `company` 字段的首尾引号。关键的发现是：`timely_response` 与 `company_response` 记录的是投诉**处理之后**的结果，若当作输入特征即构成「用答案预测答案」的泄漏，故在字典中明确标注为「结果字段，不可作输入」。

防泄漏划分（`src/split.py`）。针对阶段 1 随机划分的乐观偏差，做了三件事：① 按接收时间排序、前 80% 训练后 20% 测试，不洗牌以保序；② 消费者级去重——采用「叙事前 64 字符优先、ZIP+公司回退」的两级指纹，避免全文哈希因个别字差异而漏判、也避免单纯 ZIP+公司误删不同消费者；③ 泄漏检查输出 `leak_free` 布尔值与训练/测试时间边界。在 n=300,000 抽样上去重移除 184,222 条（61.4%），反向印证了模板化批量投诉的严重程度。

测试。针对 8.9 GB 全量数据跑错成本高的问题，将关键逻辑先行单元化（时间顺序、去重、泄漏检测、非法比例），共补 7 项，含故意构造的边界用例，全部通过后才上真实数据。

（大白话）：邵飞翔主要担心一件事——同一个人会复制同一段话、批量投诉几十家不同的公司。如果随机划分数据，这个人的记录一半掉进训练集、一半掉进测试集，模型就等于「提前偷看过答案」，分数是假的。所以他的核心工作是**把数据切干净**：先来的当训练、后来的当测试；用「叙事开头 64 个字」判断是不是同一个人复制的；最后再检查一遍，确认训练集和测试集里没有一个重复的人。

7.2 沈勇：EDA、文本基线与图表

EDA（`src/eda.py`）。坚持「有目的」的探索，只回答三个问题：类别多不平衡、哪些字段缺失严重、叙事披露率是否逐年下降。得到 Top1 类占 44.3%、Top10 占 87.4%、叙事缺失 77% 等结论，直接支撑「评估须用宏 F1 而非准确率」的判断。

文本基线（`src/text_baseline.py`）。实现 TF-IDF（1–2 gram、sublinear TF）+ 逻辑回归（`class_weight=balanced`）。实际运行中修正了 `liblinear` 不支持多分类的问题（改用 `lbfgs`），并补充「Product 单特征逻辑回归」作为元数据对照，以回答「文本特征比元数据强多少」。评估坚持复用阶段 1 自写的 `evaluate.py` 复算宏 F1，不盲信第三方库；5 个种子（42–46）重复实验标准差为 0，说明固定数据与参数下逻辑回归收敛稳定。

图表与报告。生成 fig4–fig7（类别分布、缺失率、叙事趋势、基线对比），处理了中文字体、长标签截断、数字标注对齐等细节。整理报告初稿与附录 B 提示词记录。关键观察「时间外划分后测试集叙事覆盖率骤降至 8.7%」直接改变了设计方向（纯文本不可行，须融合元数据），写在第 3.4 节。

（大白话）：沈勇的工作可以理解为「先看、再算、最后说」三步。先看数据长什么样（多少类、哪些字段空着、叙事是不是越来越少）；再用最基础的方法跑出一个结果当参照；最后画图、写报告，把结论讲清楚。他最重要的发现是：切完数据后，测试集里能用来练文本模型的样本只剩 8.7%，说明「光靠文本分类」这条路走不通，必须把文本和产品信息合起来用——这个发现直接定了后面阶段的方向。

7.3 共同完成

- 问题表述与对阶段 1 反馈的回应（第 1 节），各自起草后合并。
- 关键数字的解读——「61.4% 去重率」「8.7% 测试集叙事覆盖率」背后的含义由两人共同推敲，而非仅罗列数字。
- 消费者指纹「叙事优先、ZIP+公司回退」方案经互相质疑打磨而成。
- 报告定稿前，所有数字均在本地真实数据上重新核对。

8. 参考文献

1. Consumer Financial Protection Bureau. Consumer Complaint Database. https://www.consumerfinance.gov/data-research/consumer-complaints/
2. Joachims, T. Text categorization with support vector machines. ECML, 1998.
3. Pedregosa et al. Scikit-learn: Machine Learning in Python. JMLR, 2011.

附录 A：数据字典

数据来源：CFPB Consumer Complaint Database（公开数据，许可：公共领域 / 开放数据）。观测单位：一条消费者投诉；全量约 16,545,376 条记录，16 个字段。

#	字段名	类型	含义	示例	缺失处理
1	`date_received`	日期	投诉被 CFPB 接收的日期（ISO8601）	2024-01-25T19:56:56.000Z	缺失/非法日期整条拒绝
2	`product`	分类	产品大类（已按历史 schema 归一化）	Credit reporting or other personal consumer reports	空值记为 UNKNOWN
3	`sub_product`	分类	产品子类	Credit reporting	空值记为 UNKNOWN
4	`issue`	标签	问题类型（本项目预测目标，181 类）	Incorrect information on your report	空值记为 UNKNOWN
5	`sub_issue`	分类	问题子类	Account status incorrect	空值记为 UNKNOWN
6	`narrative`	文本	消费者自述（自由文本，平均 1020 字符）	They keep calling about a debt I do not owe.	空串表示未提供
7	`company_public_response`	文本	公司对 CFPB 的公开回应	Company has responded to the consumer...	空值记为 UNKNOWN
8	`company`	分类	被投诉公司名	Experian Information Solutions Inc.	空值记为 UNKNOWN，去除首尾引号
9	`state`	分类	消费者所在州（大写两位缩写）	TN	空值记为 UNKNOWN，统一转大写
10	`zip_code`	分类	消费者邮编（5 位）	38141	空值记为 UNKNOWN
11	`tags`	分类	标签（如 Servicemember）	Older American	空值记为 UNKNOWN
12	`submitted_via`	分类	提交渠道	Web	空值记为 UNKNOWN

13	`date_sent_to_company`	日期	投诉转交公司的日期	2024-01-25T19:56:59.000Z	缺失不拒绝
14	`company_response`	分类	公司给消费者的处理结果	Closed with non-monetary relief	空值记为 UNKNOWN
15	`timely_response`	布尔	公司是否及时响应（Yes/No）	Yes	空值记为 UNKNOWN
16	`complaint_id`	标识	投诉唯一编号	8223945	空值记为 UNKNOWN

注：`issue` 是本项目的预测目标（多类分类标签），其余字段为可用输入特征。

附录 B：生成式 AI 使用声明与提示词（Prompt）记录

B.1 使用声明

本阶段我们使用了生成式 AI 助手（WorkBuddy）辅助开发，但**所有统计数字、评估指标、代码逻辑均由我们自己在本地真实数据上运行并审查验证。**AI 的使用范围限定在：模块代码框架的起草、测试用例的启发、报告文字的组织。以下是具体的提示词设计和我们对 AI 输出的审查过程，以体现我们自己的思考与决策。

B.2 提示词设计原则

我们没有把整个项目「一句话丢给 AI」，而是把问题拆成几个可验证的小任务，每个任务配一个明确的提示词，并且对 AI 的每个输出都做了**人工审查 + 本地运行验证。**这是我们认为正确的 AI 使用方式：AI 是辅助，决策和验证仍然是我们自己做。

B.3 分模块提示词记录

Prompt 1 — 防泄漏划分（核心）

我们想问的问题：阶段 1 用随机划分，但 CFPB 数据里同一消费者会批量提交模板化投诉，导致同一人的记录可能同时出现在训练集和测试集，指标虚高。请帮我想一个防泄漏的划分方案，并给出 Python 函数框架。要求：① 按时间划分（前 80% 训练、后 20% 测试）；② 用某种「消费者指纹」去重；③ 能输出泄漏检查结果；④ 固定随机种子可复现。

我们的审查与修正：AI 最初建议用「ZIP + 公司名」做消费者指纹，我们讨论后认为：有叙事文本时用叙事前 64 字符做指纹更准（几乎能断定是同一模板），无叙事时才回退到 ZIP+公司。这是**我们自己补充的设计**，AI 没想到这一层。此外我们要求它额外输出 `leak_free` 布尔值和 train/test 的时间边界，并写了 7 个针对性单元测试确认逻辑正确后才在真实数据上跑。

Prompt 2 — EDA 要有目的

我们想问的问题：别给我一堆没用的图。请帮我设计「有目的」的 EDA：围绕三个具体问题——① 类别到底多不平衡（Top10 占比、尾部类数量）；② 哪些字段缺失率高、能不能当特征；③ 叙事披露率是不是逐年下降（用来佐证模板化投诉假设）。每个统计都要能对应报告里的一个结论。

我们的审查与修正：我们坚持「每个数字都要有结论」，删掉了 AI 建议的无关统计（比如公司名的 word cloud）。缺失率我们要求区分「影响输入的字段」和「结果字段」（timely_response 等结果字段不能当输入，否则是泄漏）。

Prompt 3 — 文本基线怎么选

我们想问的问题：阶段 2 要一个「简单基线」。我的数据是英文投诉文本，类别极度不平衡。请帮我评估：TF-IDF + 逻辑回归是否合适？参数怎么设（ngram、sublinear、class_weight）？评估要不要置信区间？请给出代码框架和理由。

我们的审查与修正：我们确认了评估指标仍然用阶段 1 自写的 `evaluate.py`（不信任 sklearn 的 macro-F1 细节，要自己复算）。我们加了一条「元数据对照」（Product 单特征逻辑回归），用来回答「文本特征到底比元数据强多少」——这是老师会问的问题，AI 最初没主动想到。

Prompt 4 — 数据字典

我们想问的问题：帮我按 16 个字段逐条生成数据字典，每条要：字段名、类型、含义、示例值、缺失处理方式。输出 Markdown 表格 + JSON，方便放报告附录。

我们的审查与修正：我们逐字段核对了示例值和缺失处理，修正了 sub_product/sub_issue 与 product/issue 的关系描述，并标注了哪些字段是「结果字段不能当输入」。

Prompt 5 — 时间泄漏强验收测试

我们想问的问题：给我写一个「会失败」的回归测试，专门防时间泄漏——断言训练集的最大日期不超过测试集的最小日期，并且测试集的日期绝不能出现在训练集里。我要让「训练集绝不用未来数据」这句口头承诺变成一个可执行、跑不过就报错的测试。

我们的审查与修正：AI 给的初版只比较了单个日期值，我们要求它补上「测试集日期绝不进入训练集」的集合级断言（`overlap=0`），并确认这个测试在故意构造的泄漏数据上会确实失败、在正确划分上会通过——用「反向验证测试本身有效」的方式，避免一个永远通过的假测试。

B.3.1 大白话对照表

Prompt	我们问了 AI 什么（大白话）	我们改了什么（大白话）
1 防泄漏划分	怎么防止同一人重复投诉导致作弊?	AI 说用「邮编+公司」判断同一人，我们觉得不准，改成「叙事前 64 字优先、没有才退回邮编+公司」
2 EDA 要有目的	别给我一堆没用的图	删掉无关统计；要求区分「能用的字段」和「结果字段」
3 文本基线怎么选	TF-IDF+逻辑回归合不合适?	自己加了「元数据对照」，回答「文本到底比产品信息强多少」
4 数据字典	按 16 字段生成字典	逐字段核对，修正父子关系，标出结果字段
5 时间泄漏测试	写个「会失败」的测试防时间泄漏	补上「测试集日期绝不进训练集」的断言，并验证测试本身有效

B.4 我们自己的思考总结

我们觉得这次最有价值的一点是：**AI 给的框架都是「教科书式」的，但真正体现我们理解的，是那些 AI 没想到、需要我们自己补的细节**——比如：① 消费者指纹的「叙事优先、ZIP+公司回退」两级策略；② 结果字段（timely_response、company_response）不能当输入，否则是「用答案预测答案」；③ 评估指标坚持自写复算，不盲信第三方库；④ 把「不泄漏」从口头承诺变成一个会失败的回归测试（Prompt 5），用测试而非声明来保证数据划分的正确性。这些是我们自己思考出来的东西，也是我们认为这份报告区别于「纯 AI 代做」的地方。

附录 C：可复现命令

```
pip install -r requirements.txt
pytest tests/ -v
python -m src.eda --data /path/to/complaints.csv --sample 300000 --seed 42
python -m src.split --data /path/to/complaints.csv --sample 300000 --seed 42
python -m src.text_baseline --data /path/to/complaints.csv --sample 200000 --seed 42
python tools/figures_phase2.py
```